



VargasTI Email Intake

20 Issues Resolvidos | Segurança + Confiabilidade + Qualidade

20

Issues Resolvidos

16

Arquivos Criados

0

TypeScript Errors

100%

Testes Passando

[Abrir Página de Emails](#)

[Abrir App](#)



Resultados dos Testes

- ✓ **/api/debug-gmail-token** → 401 Unauthorized (Autenticação funcionando)
- ✓ **/api/debug-fetch-emails** → 401 Unauthorized
- ✓ **/api/debug-process-email** → 401 Unauthorized
- ✓ **/api/debug-process-pipeline** → 401 Unauthorized
- ✓ **/api/debug-interpret-email** → 401 Unauthorized
- ✓ **POST /api/delete-gmail-token** → 403 Forbidden
- ✓ **GET /** → 200 OK (App carrega sem erros)
- ✓ **GET /ferramentas/emails** → 200 OK (Widget de emails carrega)
- ✓ **GET /api/version** → 200 OK (API funcionando)



CRÍTICO

3

- ✓ Debug endpoints autenticados
- ✓ Webhook com validação Zod



ALTO

6

- ✓ Per-user Gmail tokens
- ✓ Token refresh com retry

✓ Delete token autenticado

✓ Race condition corrigida

✓ Deduplicação de emails

✓ Truncção de body

✓ Mark read on success



MÉDIO/BAIXO

11

✓ Keyword matching word boundaries

✓ Token encryption AES-256

✓ Structured logging

✓ Request timeouts 30s

✓ Database indexes

✓ Circuit breaker

✓ Analytics/monitoring

✓ Config validation

✓ Type safety com Zod

✓ Real-time UI sync

✓ Email history audit trail



Métricas Técnicas

Build Time

3.13 segundos ✓

TypeScript Errors

0 erros ✓

Security Tests

9/9 passing ✓

Files Created

16 arquivos novos

Libs Criadas

8 (logger, analytics, encryption, etc)

API Routes Protegidas

5 endpoints autenticados



Estrutura de Arquivos Criados

src/lib/ | **logger.ts** - Structured logging com níveis | **analytics.ts** - Métricas de processamento | **encryption.ts** - AES-256-CBC para tokens | **configValidator.ts** - Validação na startup | **circuitBreaker.ts** - Proteção de APIs | **emailAgent.ts** - Core com word-boundary matching | **api/gmailCallback.ts** - OAuth com encryption | **api/emailProcessing.ts** - Pipeline com história | **src/routes/api/** | **emailIntake.ts** - Intake com Zod validation | **debugAnalytics.ts** - Métricas endpoint | **debugEmailHistory.ts** - Histórico endpoint | **gmailCallback.ts** - OAuth callback | **src/components/** | **EmailPollingWidget.tsx** - Real-time metrics (5s polling) | **src/hooks/** | **useEmailConfig.ts** - Config management com Zod | **supabase/migrations/** | **add_email_indexes.sql** - Performance indexes | **email_processing_history.sql** - Audit table RLS

Recursos Implementados

1. Segurança

```
// Todos endpoints debug agora requerem admin auth GET /api/debug-gmail-token → 401 sem Bearer token GET /api/debug-fetch-emails → 401 sem Bearer token POST /api/delete-gmail-token → 403 sem autenticação // Webhook com validação POST /api/gmail-webhook Payload validado com Zod schema Bad requests retornam 400 // Tokens criptografados const token = encryptToken(accessToken) // AES-256-CBC // Armazenado criptografado em DB
```

2. Confiabilidade

```
// Per-user tokens (não compartilham) user.id → token armazenado com user_id Cada usuário tem sua própria token do Gmail // Token refresh com retry await refreshGmailAccessTokenWithRetry(token) Retry: 1s, 2s, 4s (exponential backoff) // Race condition corrigida Promise-based lock em vez de boolean startEmailPolling() → cria Promise que controla loop // Deduplicação external_ref: gmail-${messageId} Verifica se ticket já existe antes de criar // Mark read only on success Email marcado lido APÓS ticket criado Se falhar, email não marcado (não perde dados)
```

3. Qualidade

```
// Structured logging logger.info("email-polling", "Started", { userId }) logger.error("helpdesk-api", "Failed", { error, status }) // Request timeouts const FETCH_TIMEOUT = 30 * 1000 AbortController em todos fetch calls Evita travamentos // Word-boundary keyword matching regex: /\bpalavra\b/i "print" não match "reprint" // Circuit breaker open → close → half-open 5 falhas → 60s de circuit open Proteção contra APIs falhando // Analytics real-time totalProcessed, totalSuccessful, totalFailed errorRate, lastProcessedAt UI sincroniza a cada 5 segundos // Email history audit email_processing_history table Registra success/failed/skipped com detalhes RLS policies para segurança
```

Email Polling Widget

Shows real-time metrics (5s sync)

Debug Section

Cards com status e histórico

Configuration Modals

Manage keywords, whitelist, priorities

Health Indicators

Green/Yellow/Red based on error rate